

A GPU Branch and Bound for the Quadratic Assignment Problem

Design and implementation of `cgbncudaqapbb.cu`

31 August 2026

This document describes the design and implementation of `cgbncudaqapbb.cu`, an exact GPU branch-and-bound solver for the Quadratic Assignment Problem. The program is part of a repository containing three brute-force QAP solvers (`qap`, `cudaqap`, `cgbncudaqap`), a serial branch-and-bound solver (`qapbb`), and a random-instance generator. The software is © 2025–2026 Stefan Teleman and is released under the MIT License.

Section 13 credits every algorithm the implementation uses, together with where in the source each one appears. Nothing in this program is original mathematics; the contribution is the arrangement.

Contents

1	Introduction	2
1.1	What the program is	2
1.2	Where it sits among the solvers	2
1.3	Reading guide	2
2	The problem	2
2.1	Formulation	2
2.2	Why it is hard	3
2.3	Two instance families used throughout	3
3	Inherited structure	3
3.1	From <code>cgbncudaqap.cu</code>	3
3.2	From <code>qapbb.c</code>	3
3.3	What was dropped, and why	4
4	The bound	4
4.1	Root reduction	4
4.2	Minimal scalar products	5
4.3	The linear subproblem	5
4.4	Carrying the committed cost	6
4.5	The last two facilities	6
5	The search tree	6
5.1	Why the original branching rule could not be kept	6
5.2	Fixed-order placement	6
5.3	Root relabelling	7
5.4	The shape of the DFS	7

6	Parallel decomposition	7
6.1	Why splitting at a fixed depth fails	7
6.2	Phase one: breadth-first frontier growth	8
6.3	Phase two: persistent threads, depth-first	9
6.4	What it bought	9
7	The incumbent	9
7.1	One variable for two jobs	9
7.2	The starting bound	9
7.3	A feasible solution from the bound, for free	10
7.4	What is reproducible and what is not	10
8	Arbitrary-precision accounting	10
8.1	The check that had to be replaced	10
8.2	The replacement	11
8.3	Why the sum needs CGBN, and the counts do not	11
8.4	The reduction kernels	11
8.5	The signed residual	12
8.6	Validating a path a correct run cannot reach	12
9	Memory and launch geometry	12
9.1	The per-thread workspace	12
9.2	Why global memory and not shared	13
9.3	Choosing the launch	13
9.4	Frontier buffers	13
10	Limits and guards	13
11	Verification	14
11.1	Oracles	14
11.2	compute-sanitizer	14
11.3	Two bugs the self-check did not catch	15
12	Measurements	15
12.1	The largest run	16
13	Algorithm credits	16
13.1	The problem itself	16
13.2	Branch and bound	16
13.3	The linear assignment subproblem	17
13.4	Sorting	17
13.5	Ranking and unranking permutations	17
13.6	Heuristics	18
13.7	Parallel search	18
13.8	Arbitrary-precision arithmetic	18
13.9	Data and tools	19
13.10	From elsewhere in this repository	19
14	References	19
A	Source map	21
B	Output fields	22

1 Introduction

1.1 What the program is

`cgbncudaqapbb` solves the Quadratic Assignment Problem exactly, on a single NVIDIA GPU, by branch and bound. It takes the same input files as the other solvers in this repository — either a flow/distance file pair (`-f/-d`) or a QAPLIB-style single file (`-s`) — and reports the minimum cost, a minimizing assignment, and a set of counters that let the run check itself.

It exists because the three brute-force solvers in the repository stop at $N \approx 16$. They enumerate all $N!$ permutations, which is $O(N! \cdot N^2)$ work; on the GPU that is about 0.1 s at $N = 12$, thirty seconds at $N = 14$, and two and a half hours at $N = 16$. Pruning removes the factorial from the runtime, and the difference is not a constant. Extrapolating `cudaqap`'s measured rate at $N = 16$ — $16!$ permutations in 8511 seconds, with the per-permutation cost growing as N^2 — brute force would need some 5×10^8 years at $N = 25$, where `cgbncudaqapbb` finishes in 2.5 seconds; 10^{16} years at $N = 30$, where it finishes in twenty; and 10^{19} years at $N = 32$, the largest instance run to completion here, in 21 minutes.

1.2 Where it sits among the solvers

program	method	hardware	reach
<code>qap</code>	enumerate all $N!$	CPU, serial	$N \leq 12$ in practice
<code>cudaqap</code>	enumerate all $N!$	GPU	$N \leq 16$; refuses $N > 20$
<code>cgbncudaqap</code>	enumerate all $N!$	GPU	$N \leq 16$; refuses $N > 24$
<code>qapbb</code>	branch and bound	CPU, serial	instance-dependent
<code>cgbncudaqapbb</code>	branch and bound	GPU	instance-dependent

`cgbncudaqapbb` is a hybrid of the last two. Its scaffolding — option parsing, input readers, launch-geometry selection, the packed argmin key, 128-bit ranks, the CGBN accumulators — comes from `cgbncudaqap.cu`. Its bound comes from `qapbb.c`. Its tree and its parallel decomposition belong to neither, for reasons developed in Sections 5 and 6.

1.3 Reading guide

Sections 2–4 are the mathematics the program implements, none of it new. Sections 5–8 are the engineering, which is where the decisions are. Section 10 lists what the program refuses to do and why; Section 11 describes how it was checked, including two bugs that produced correct-looking output. Section 13 is the attribution list.

2 The problem

2.1 Formulation

In the Koopmans–Beckmann form of the QAP, n facilities are to be placed on n locations. $A = (a_{ij})$ gives the flow between facilities i and j ; $B = (b_{kl})$ gives the distance between locations k and l . A placement is a permutation φ of $\{1, \dots, n\}$ sending facility i to location $\varphi(i)$, and its cost is

$$Z(\varphi) = \sum_{i=1}^n \sum_{j=1}^n a_{ij} b_{\varphi(i)\varphi(j)} + \sum_{i=1}^n c_{i\varphi(i)},$$

where the optional linear term $C = (c_{ik})$ is the cost of placing facility i at location k on its own. The problem is to minimise Z over all $n!$ permutations. The repository's inputs supply A (files named `fldata-*`) and B (`dstdata-*`) and leave $C = 0$; the linear term is nonetheless

carried all the way through the implementation, because the bound machinery generates one whether the input had one or not.

2.2 Why it is hard

The QAP is NP-hard, and remains so in a strong sense: no polynomial-time algorithm can guarantee a constant-factor approximation unless $P = NP$. Exact solution of general instances of size 30 is a serious computation, and instances in the thirties are where the published exact literature lives. That is the scale this program is aimed at.

2.3 Two instance families used throughout

- **QAPLIB chr instances** (`chr12a`, `chr20a`, `chr25a`), whose optima are published as 9552, 2192 and 3796. These are the correctness oracle: any change that stops reproducing them is wrong.
- **Random pairs** generated by `genrandomdata` (`fldata-k` / `dstdata-k`). These have widely spread entries, which makes the bound strong and the instances comparatively easy; the `chr` instances are structured and much harder per unit of size.

3 Inherited structure

3.1 From `cgbncudaqap.cu`

The following are carried over essentially unchanged, and are documented in `cudaqap.md` rather than here:

- `getopt_long` option handling with a static `long_options` table; the three input readers (pair format, single-file format, and the shared tokeniser); `Verify()`; the vector printers used by `-p`.
- `Timestamp()` and `PrintTimediff()`, the labelled variant, since two intervals are reported (the search, and the arbitrary-precision reduction that follows it).
- The launch-geometry search: individual `cudaDeviceGetAttribute` queries rather than `cudaGetDeviceProperties` (which costs $\sim 700\mu\text{s}$ on this machine and inflates the measured time of a short kernel), and `cudaOccupancyMaxActiveBlocksPerMultiprocessor` to score candidate block widths.
- The packed argmin key: a single `uint64_t` holding `(cost << 32) | threadId`, so that a plain 64-bit `atomicMin` selects the lowest cost and breaks ties on the lowest thread.
- 128-bit arithmetic for permutation ranks, and the CGBN accumulator pattern with its two-stage reduction kernels.
- `BigToString()`, which converts a CGBN value to decimal through GMP's `mpz_import` and `mpz_get_str`.

3.2 From `qapbb.c`

`qapbb.c` is a literal C translation of a FORTRAN IV branch-and-bound QAP program: one C function per FORTRAN subprogram, the original statement labels preserved as C labels (`L5210`, `L2900`, ...), the original variable names, and 1-based arrays with element 0 allocated and unused so that every subscript expression matches the Fortran.

Four of its subprograms are ported to the device, into namespace `qapbb`, with that character intact — 1-based arrays, FORTRAN label numbers, original names lower-cased:

FORTTRAN	device function	role
SSORT	<code>qapbb::ssort</code>	Shell sort, carrying a companion array
LSAP	<code>qapbb::lsap</code>	linear sum assignment, shortest augmenting path
WEGSPE	<code>qapbb::wegspe</code>	sorted row tables for the scalar products
PROGNO	<code>qapbb::progno_level</code>	the minimal scalar products themselves

The bodies of `ssort`, `lsap` and `wegspe` differ from `qapbb.c` only in their signatures — `int32_t` for `int`, pointer syntax, and the `__device__` qualifier. A normalised `diff` against `qapbb.c` shows no other change, and keeping it that way is deliberate: it is the only practical way to be confident that a thousand lines of translated Fortran are still doing what they did.

The root reduction (the preamble of subroutine QAP) is ported to the host as `RootReduce`, likewise verbatim, because its result is identical for every thread.

3.3 What was dropped, and why

- ALTKOS, which computes alternative costs to choose the branching pair, and with it the ZUL/IKAP bookkeeping that supports that branching rule. Section 5 explains why.
- PROGNO’s CSPEI cache. The original stores each level’s minimal scalar products so that backtracking can read them back instead of recomputing them. That pays only when a level’s matrix is revisited unchanged, which is exactly what the original’s binary “forbid (i, j) ” branch does and what the tree used here never does: coming back up, the next child is a different assignment and needs a different matrix. Dropping the cache removes $n(n+1)(2n+1)/6$ words from the per-thread workspace — 11 KB at $n = 20$ — which on a GPU is the difference between running and not.
- The QAP driver itself, which is replaced entirely.

4 The bound

Everything the search does rests on being able to say, cheaply, that no completion of a partial placement can beat the best solution found so far. The bound used is the Gilmore–Lawler bound, in the form Boenniger, Burkard and Stratmann implemented it.

4.1 Root reduction

Before any search, A , B and C are transformed once, on the host, by `RootReduce`. Three things happen, in order:

1. The diagonal terms move into the linear term: $c_{ij} += a_{ii}b_{jj}$. A facility’s self-flow multiplied by a location’s self-distance depends only on the pair (i, j) , so it is linear and belongs in C . The diagonals of A and B are then set to a large sentinel so they take no part in the minima that follow.
2. A and B are reduced row-wise: the minimum of each row is subtracted from it, and the resulting constant is compensated exactly into C . The compensation is $c_{ij} += u_i \sum_{k \neq j} b_{jk} + v_j \sum_{k \neq i} a_{ik}$ with u, v the row minima of A and B .
3. The same again column-wise. Afterwards the diagonals are zeroed.

The transformation is exact: for any complete permutation, the objective computed from the reduced A , B , C equals the objective computed from the originals. The point is that the reduced A and B are non-negative with small entries, which makes the bound of the next subsection much tighter.

The program does not take this on trust. The reported assignment is re-scored by `HostObjective` against the untouched input matrices, and a disagreement with the search's own figure is reported on standard error.

4.2 Minimal scalar products

At a node where a set F of facilities is still unplaced and a set G of locations still free, the interaction between two *unplaced* facilities is not yet determined. The Gilmore–Lawler bound underestimates it, one facility at a time.

Consider placing facility $i \in F$ at location $j \in G$. Its remaining interactions are $\sum_{k \in F \setminus \{i\}} a_{ik} b_{j\varphi(k)}$ for whatever φ turns out to be. Whatever φ is, that sum is a scalar product of the multiset $\{a_{ik}\}$ with the multiset $\{b_{jl}\}$ under *some* pairing. The rearrangement inequality says that over all pairings, the minimum is attained by sorting one multiset in decreasing order, the other in increasing order, and pairing them off in that order:

$$\min_{\sigma} \sum_{k=1}^m a_{(k)}^{\downarrow} b_{(\sigma(k))} = \sum_{k=1}^m a_{(k)}^{\downarrow} b_{(k)}^{\uparrow}.$$

That minimum is a valid lower bound on the interaction term, and it is computable without knowing φ .

`qapbb:wgspe` maintains the two sorted lists. At the root it sorts every row of A decreasingly and every row of B increasingly (excluding the diagonal) into `aspei` and `bspei`. Below the root it does not sort again: descending one level, it copies level k 's segment to level $k+1$'s with exactly one occurrence of the just-placed row's and column's contribution deleted from each remaining row. Level k 's segment is left intact, which is what lets the search walk back up without recomputing anything. `qapbb:progn_level` then forms, for every (i, j) still free, the scalar product of i 's descending row against j 's ascending row.

Segments are addressed by a table `VEKSUM`, where `VEKSUM[L]` is the offset of level L 's block; a level- L block holds $(n-L)$ rows of $(n-L-1)$ entries, so the whole structure is $n(n+1)(2n-2)/6$ words.

4.3 The linear subproblem

Adding the linear term to the minimal scalar products gives a matrix

$$M_{ij} = \underbrace{c_{ij}}_{\text{linear}} + \underbrace{\sum_k a_{ik}^{\downarrow} b_{jk}^{\uparrow}}_{\text{Gilmore-Lawler}} + \underbrace{a_{ir} b_{cj} + a_{ri} b_{jc}}_{\text{new pair } (r,c)}$$

over the free i and j , where (r, c) is the pair this node has just fixed. Every completion of the node assigns each free facility to a distinct free location, so its cost is at least the cost of the cheapest such assignment — a linear sum assignment problem on M .

`qapbb:lsap` solves it by the shortest augmenting path method: build an initial partial assignment greedily, then repeatedly take an unassigned row, compute shortest alternating-path distances in the reduced costs, augment along the shortest path, and update the dual variables. It returns the optimal value z^* and, incidentally, an optimal assignment — which Section 7 puts to a second use.

The bound at the node is $z_{\text{part}} + z^*$, where z_{part} is the cost already committed by the placements on the path from the root.

4.4 Carrying the committed cost

z_{part} and the linear term C are maintained incrementally. When the search commits to placing facility r at location c :

$$z_{\text{part}} += c_{rc}, \quad c_{ij} += a_{ri}b_{cj} + a_{ir}b_{jc} \quad \text{for all still-free } i, j.$$

The second line is **ShiftC**, and backing out subtracts exactly the same quantity, so C is restored bit for bit. It is the one piece of state the depth-first search mutates in place, and it is why every thread needs a private copy of C rather than sharing the root's.

The update is *deferred*: a tentative child computes its bound with $C + \Delta$ folded into M but leaves C alone, and only pays for the $O(n^2)$ update if it survives the bound test. This is the original's arrangement and it is worth keeping — most children do not survive.

4.5 The last two facilities

When only two facilities and two locations remain, the bound machinery would be solving a 2×2 assignment problem to decide between two possibilities. The original evaluates both in closed form instead, and so does **LeafPair**: with free facilities i_1, i_2 and free locations j_1, j_2 ,

$$Z = z_{\text{part}} + c_{i_1j_1} + c_{i_2j_2} + a_{i_1i_2}b_{j_1j_2} + a_{i_2i_1}b_{j_2j_1}$$

and the same with j_1, j_2 exchanged. A consequence worth noting: the LSAP is therefore never asked for a subproblem smaller than 3×3 .

5 The search tree

5.1 Why the original branching rule could not be kept

qapbb branches on a single pair. At each node it computes, for every candidate assignment (i, j) , an “alternative cost” — roughly, what it would cost to be forced away from (i, j) — and branches on the pair with the largest one: the left child assigns (i, j) , the right child forbids it. This is a strong rule. It is also unusable on a GPU, for a reason that has nothing to do with its quality.

A node in that tree is identified by a *set of forbidden cells* together with a partial assignment. There is no cheap index for such a thing. To hand a thread a subtree you must either ship it the set, or ship it a path and have it replay the alternative-cost computation at every level to reconstruct which branch was taken — which costs as much as the subtree. There is no way to say “thread 8000 takes the k -th subtree”.

5.2 Fixed-order placement

The tree used here places facilities in a fixed order determined once at the root: level L places facility L , and its children are the still-free locations in increasing order. A node at level L is therefore an injective L -tuple of locations — a partial permutation — and nothing else. Two consequences follow immediately:

- A node has an **index**. Numbering the children of a level- k node $m = 0, 1, \dots, n - k - 1$ in increasing order of location, the rank of a node satisfies

$$\text{rank}_L = \text{rank}_{L-1} \cdot (n - k) + m, \quad k = L - 1,$$

which is an ordinary mixed-radix number with radix $n - k$ at level k . This is the factorial number system, restricted to L digits.

- A node can be **rebuilt** from that index alone. **Descend** peels the digits off the bottom by repeated division, then walks down from the root applying **wegspe** and **ShiftC** at each level. Because nothing on that path needs a bound, the LSAP is never called, and the walk costs $O(L \cdot n^2)$ rather than $O(L \cdot n^3)$.

The price is a weaker branching rule, and it is a real price: on the same instance with the same bound, this tree is larger than **qapbb**'s. It is also the one that can be handed out, and there are fifteen thousand threads to hand it to.

5.3 Root relabelling

Since index order is now branching order, the order matters. **ReorderFacilities** relabels the facilities at the root by decreasing total flow $\sum_j (a_{ij} + a_{ji})$, so the most heavily connected facility is placed first and the bound tightens as early as possible. B is untouched — only the facilities are relabelled — and the solution is mapped back through the permutation before it is reported.

This is not cosmetic. Measured on a single-threaded host build of the same search, so that the node count is not perturbed by the parallel incumbent:

instance	identity order	flow order
chr20a	32,462	11,859
chr25a	17,780,726	826,674
fldata-400	178,244	100,979

5.4 The shape of the DFS

RunSubtree is an explicit-stack depth-first search — no recursion, which would be expensive in local memory and unnecessary given the loop structure the Fortran already had. Per level it keeps a cursor into the free-location list (**cur**), the location chosen (**colsel**), and the partial solution (**sol**); the arrays live in the thread's workspace. The loop is:

```

for (;;) {
  if (cursor at level k exhausted) {
    if (k == Depth) break;           // subtree finished
    k--; undo ShiftC and zpart; unmark; // backtrack
    continue;
  }
  c = next free location; r = k + 1; L = k + 1;
  mark r, c; zpc = zpart + C(r,c);
  if (L == N - 2) { LeafPair(...); Account(L); unmark; continue; }
  if (ChildBound(...) >= CurrentBound()) { Account(L); unmark; continue; }
  ShiftC(forward); zpart = zpc; k = L; // descend for good
}

```

6 Parallel decomposition

6.1 Why splitting at a fixed depth fails

The obvious decomposition, once nodes have indices, is to cut the tree at some depth D , index the $n!/(n-D)!$ nodes there, and let threads claim ranges of them from a work queue. That was the first working version of this program. It was correct, and it was slower than a single CPU core.

The reason is in its own output. On the 25×25 instance **fldata-625**, with a depth-5 cut:

B&B nodes:	8675014
Busiest:	257352 node(s)
GPU time:	61.960711364 second(s).

8.68 million nodes across the launch, and one thread held 257,352 of them. Every other thread finished in the first second and waited. Under a pruning bound the subtree sizes span five orders of magnitude, and no choice of D repairs that: cutting finer makes the queue longer without making the largest piece smaller, because the piece that matters is deep inside one branch. Two further mechanisms made the fixed-depth version worse than it looks:

- **Spine replay.** A task at depth D must walk down to its range, which re-evaluates the same shallow nodes once per task. At $N = 20$ this alone turned a 101-thousand-node search into 588 thousand.
- **A cache does not fix it either.** A node's bound depends only on the partial placement, never on the incumbent, so shallow bounds can be memoised. Adding that cache helped ($588 \rightarrow 140$ thousand nodes) but introduced a stampede of its own: tens of thousands of threads all start at once and all miss the same twenty level-1 nodes. Instrumented at $N = 20$, the twenty level-1 nodes were computed 138,561 times and the 380 level-2 nodes 255,469 times. A priming pass fixed the stampede, and the tail was still 6248 nodes in one thread out of 140 thousand.

None of that machinery survives in the shipped program. It is recorded here because the alternative — growing the frontier — looks like an unnecessary complication until you have watched the simple thing fail.

6.2 Phase one: breadth-first frontier growth

Instead of guessing where to cut, the program grows a frontier of *live* nodes until it has enough of them.

`ExpandFrontier` takes the live nodes at level L as an array of ranks. One thread per node: it rebuilds the node's state from its rank with `Descend`, bounds all $n - L$ children, accounts for the dead ones, and appends the survivors' ranks to the next level's buffer with a single `atomicAdd` on a shared counter. The host then swaps the buffers and repeats. No frontier node depends on any other, so there is no synchronisation inside a level at all — only the kernel boundary between levels.

The loop stops when any of the following holds:

1. the frontier reaches 32 live nodes per thread (`-k` overrides);
2. the frontier is empty, meaning the search finished breadth-first;
3. the next level would be the leaf level;
4. the next level's children could not fit in the frontier buffer;
5. the 128-bit rank would overflow at the next level.

Condition (4) is checked as $NF \cdot (n - Depth) \leq FCap$ before expanding, which makes overflow impossible rather than merely unlikely; the kernel still counts past the end without writing, so that a broken invariant becomes a diagnostic rather than a memory stomp. Condition (5) is a running falling-factorial product against 2^{128} .

Condition (2) is the common case at small sizes. Every instance in this repository below $N = 32$ is finished by the breadth-first phase alone, and says so: `Frontier: 0 live node(s) at depth 23 - solved breadth-first.`

6.3 Phase two: persistent threads, depth-first

`BranchAndBound` is a persistent-thread kernel. Each thread loops:

```
for (;;) {
    Lo = atomicAdd(&GNEXT, QChunk);
    if (Lo >= NF) break;
    for (I = Lo; I < min(Lo + QChunk, NF); ++I)
        RunSubtree(S, Front[I], Depth);
}
```

with `QChunk = 1`: one frontier node per task. The queue hands out real work rather than mostly-dead ranges, because everything in the frontier is live by construction, and there are enough tasks that the largest is a small fraction of the whole.

6.4 What it bought

Same instance, same machine, after the change:

	fixed depth-5 split	grown frontier
nodes	8,675,014	8,596,072
busiest thread	257,352	795
GPU time	61.96 s	2.74 s

At $N = 32$, where phase two actually runs, the frontier stops at depth 7 with 665,914 live nodes and the busiest of the fifteen thousand threads carries 2,366,935 nodes out of 1,484,193,064 — 0.16% of the search.

7 The incumbent

7.1 One variable for two jobs

The best solution found so far serves two purposes: it is the answer, and it is the threshold every thread prunes against. The program keeps one `__device__ uint64_t` for both:

$$\text{MCK} = (\text{cost} \ll 32) \mid \text{threadId}.$$

A single 64-bit `atomicMin` on it selects the lowest cost, breaks ties on the lowest thread id, and simultaneously publishes a tighter pruning threshold to every other thread. `CurrentBound()` is a volatile load of the high half. There is no second broadcast and no second atomic.

The volatile qualifier matters in a way that is easy to miss: a thread reading a stale incumbent is not incorrect, it merely does more work, silently. That is a performance bug that no test would catch.

The initial value is $(\text{heuristic cost} \ll 32) \mid 0\text{xffffffff}$. No thread can own the id `0xffffffff`, so a key still carrying it at the end means no thread improved on the heuristic — in which case the search has proved the heuristic optimal, and the host reports the heuristic’s own permutation.

7.2 The starting bound

`StartBound` runs on the host before the GPU is touched. It takes the identity permutation and each of its $n - 1$ cyclic shifts (capped at 32 starts), drives each to a 2-opt local optimum by first-improvement pairwise exchange, and keeps the best. There is no randomness: two runs on the same input start from the same bound.

A branch-and-bound search with no incumbent spends its first phase proving things it could have been told, and on a GPU that is worse rather than better, because every thread does it at once. On the random instances in this repository the 2-opt bound is frequently already optimal, and the GPU run is then purely the proof.

7.3 A feasible solution from the bound, for free

The LSAP solved at every node returns an optimal assignment of the free facilities to the free locations. That assignment is a complete, feasible solution to the original problem. Scoring it costs $O(m^2)$ against the bound’s $O(m^3)$ and needs nothing the workspace does not already hold:

$$Z = z_{\text{part}} + \sum_{i \in F} \tilde{c}_{i\varphi(i)} + \sum_{\substack{i,j \in F \\ i \neq j}} a_{ij} b_{\varphi(i)\varphi(j)},$$

computed in the reduced arithmetic, which gives the true objective. `TryCompletion` does this and offers the result to the incumbent.

It matters more here than it would on one core. A sequential search reaches a leaf almost at once and prunes with a real solution from then on; fifteen thousand threads starting simultaneously have nothing but the host’s bound until one of them gets to the bottom, and everything they do in the meantime is speculative. On `chr25a`, where the host heuristic comes in at 5062 against an optimum of 3796, adding it took the node count from 19.7 million to 6.0 million.

It runs on every eighth live node rather than every one. Unthrottled it cost 28% on an instance whose incumbent was already optimal and had nothing to gain; sampled, `chr25a` settles at about 7.9 million nodes — most of the benefit for a few percent.

7.4 What is reproducible and what is not

Threads share one incumbent, so how much speculative work gets done before someone lowers it depends on timing. The node count is therefore *not* reproducible: `chr25a` measured 7,915,417, 7,939,937 and 10,001,210 nodes on three consecutive runs. All three returned 3796 with a residual of zero.

The optimum, the accounted permutation total and the residual are reproducible always. Instances whose starting heuristic is already optimal are reproducible in the node count too, because the incumbent then never moves: `fldata-625` gives 8,596,072 nodes every time.

One consequence must not be papered over. Pruning is bound $\geq$ incumbent, so a second permutation of equal cost is discarded rather than compared, and there is no equivalent of the brute-force solvers’ guarantee of returning the lexicographically first among ties. In practice the assignment matched `qap` exactly on all eight small pair-format inputs tested, but that is a property of those inputs and must not be asserted in a test.

8 Arbitrary-precision accounting

8.1 The check that had to be replaced

The three brute-force solvers prove they covered the space by counting: the iteration total must equal $N!$ exactly. That is what catches a thread that searched the wrong range or a tiling that overlapped. A pruning search visits nothing like $N!$ nodes, so the check is gone — and something has to take its place, or the program has no self-check at all beyond “the answer looks right”.

8.2 The replacement

Every node the search *disposes of* — pruned by the bound, or evaluated as the closed-form pair of leaf completions — stands for exactly $(N - L)!$ complete permutations, where L is its level. A node that is not disposed of passes its $(N - L)!$ down to its $N - L$ children, each worth $(N - L - 1)!$, and $(N - L) \cdot (N - L - 1)! = (N - L)!$. The telescoping is exact, so

$$\sum_L \text{count}_L \cdot (N - L)! = N!$$

and nothing else. This constrains the pruning as well as the enumeration: a subtree searched twice, or skipped, shows up here.

Each thread keeps one `uint64_t` count per level in its own slice of GLPR; `Account` increments it. Two kernels then finish the job.

8.3 Why the sum needs CGBN, and the counts do not

$N!$ passes 2^{64} at $N = 21$. At $N = 25$ it is 1.55×10^{25} , at $N = 30$ it is 2.65×10^{32} (108 bits), and at $N = 32$ it is 2.63×10^{35} (118 bits). No scalar a thread can hold could carry the total.

But a CGBN value is not a scalar. Its limbs live across TPI cooperating lanes, so a single thread can neither hold one nor update one atomically. That splits the arithmetic cleanly, and the split is the design:

- **Per-thread quantities stay 64-bit.** The counts are counts of nodes; no instance whose workspace can be allocated can have 2^{64} of them.
- **The one genuinely global quantity is formed cooperatively,** once, after the search, by a kernel that does nothing else.

8.4 The reduction kernels

The geometry is $\text{TPI} = 32$ and $\text{BITS} = 1024$: one warp per CGBN instance, one 32-bit limb per lane, no padding since $\text{BITS}/32 \bmod \text{TPI} = 0$. `blockDim.x` must be a whole number of instances, because CGBN lays an instance's lanes out along `threadIdx.x` and derives its warp sync mask from that. 1024 bits holds every factorial up to 170!.

AccumulateIterations / CombineIterations

Sum the per-thread node counts into one 1024-bit total. 64×256 threads give 512 one-warp instances, each summing a strided share; a single instance then folds the 512 partials. Integer addition is associative, so the order does not affect the result — the total is deterministic.

SumCounters

Collapses the per-thread per-level disposal counts to one count per level with ordinary 64-bit atomics. There are at most $N + 1$ distinct counters and no count can approach 2^{64} , so nothing here needs to be wide.

CombinePermutations

A single instance forms $\sum_L \text{count}_L \cdot (N - L)!$ in 1024-bit arithmetic and subtracts it from $N!$. The factorials are built on the host by GMP into `cgbn_mem_t` limb layout. `cgbn_mul` returns the low half of the product, which is the whole product here: a count below 2^{64} times a factorial below 2^{1024} could only need more than 1024 bits at sizes whose workspace could never be allocated.

8.5 The signed residual

The residual $N!$ —accounted is formed as a **signed** 1024-bit value, using this CGBN fork’s signed extension: `cgbn_signed_is_negative` for the sign and `cgbn_signed_abs` for the magnitude. The two are stored separately and printed as `Residual:`.

The reason is not decoration. In unsigned arithmetic an over-count would wrap $N!$ —accounted into an enormous positive number and read as a wildly wrong *under*-count — the same failure, reported backwards. Signed, the sign says which way the error went, which is the difference between a usable diagnostic and a misleading one.

8.6 Validating a path a correct run cannot reach

A correct run always has a residual of zero, so the signed arithmetic never executes in anger. It was validated the way this repository validates its other unreachable paths: `CombinePermutations`, `CgbnSetUi64` and `BigToString` were extracted verbatim into a scratch harness and driven with hand-built level counts at $N = 25$.

level counts	accounted	residual
25 nodes at level 1 (exact)	15511210043330985984000000	0
one level-1 node lost	14890761641597746544640000	+24!
one extra level-2 node	15537062060069870960640000	−23!
one extra level-1 node	16131658445064225423360000	−24!

The exact case is $25 \cdot 24! = 25!$. The three errors come back at the right magnitude and, for the two over-counts, with the right sign.

9 Memory and launch geometry

9.1 The per-thread workspace

Every thread runs an independent branch-and-bound search and needs its own copy of everything the serial program had. `LayoutWork` carves one contiguous slice out of a single managed allocation:

field	size (words)	role
<code>C</code>	n^2	the working linear term, mutated along the path
<code>umspei</code>	$n^2 + 2$	the LSAP cost matrix for the current node
<code>aspei</code>	NAB	rows of A sorted decreasing, per level
<code>bspei</code>	NAB	rows of B sorted increasing, per level
<code>vekt, h2</code>	$n + 3$ each	<code>wegspe</code> scratch and <code>ssort</code> companion
<code>ys, yt, zeile, spalte</code>	$n + 3$ each	<code>lsap</code> working vectors
<code>dminus, dplus, vor, label</code>	$n + 3$ each	<code>lsap</code> working vectors
<code>boolv, bool1</code>	$n + 3$ each	placed facilities / occupied locations
<code>cur, colsel, sol</code>	$n + 3$ each	the DFS stack
<code>frow, fcol</code>	$n + 3$ each	free lists for <code>TryCompletion</code>

with $\text{NAB} = n(n + 1)(2n - 2)/6 + 2$. The total is 48 KB per thread at $n = 25$ and 98 KB at $n = 32$.

`LayoutWork` is `__host__ __device__`. The host calls it with a null `BBWork*` purely to obtain the size; the device calls it with a real one to carve the slice. The two therefore cannot disagree about the layout, which is the kind of duplication that otherwise goes wrong quietly.

9.2 Why global memory and not shared

48 KB per thread is roughly half a block’s entire shared-memory budget on this device, for one thread. Shared memory is out of the question, and the workspace lives in global memory instead. Two consequences:

- `cgbncudaqap`’s `-m/--shmem-size` has nothing to size and is replaced by `-t/--max-threads` and `-k/--frontier`.
- `compute-sanitizer racecheck`, which instruments shared-memory accesses, has almost nothing to watch: it costs $16\times$ here against roughly $7600\times$ on `cgbncudaqap`.

9.3 Choosing the launch

Block width is chosen by stepping candidates by the warp size and scoring each with `cudaOccupancyMaxActiveBlocksPerMultiprocessor`, taking the width with the most resident threads and never a width that reports zero resident blocks (such a width cannot launch at all — a real failure mode, since the guard was added after a launch died with `cudaErrorInvalidValue`). The shared-memory argument is zero, so occupancy here is decided by registers alone.

The thread count is then the lesser of one full occupancy wave and what memory allows:

$$NTH = \min \left(\text{blocksPerSM} \cdot \text{SMs} \cdot \text{blockThreads}, \frac{\frac{3}{5} \text{ free} - \text{frontier buffers}}{\text{workspace per thread}} \right),$$

rounded down to a whole number of blocks and capped by `-t`. On the measurement machine the occupancy term binds at every size tested ($NTH = 14848$); the memory term does not bind until roughly $n = 50$, where the $O(n^3)$ sorted-row tables reach 618 KB per thread.

9.4 Frontier buffers

Two buffers of 8,388,608 ranks, ping-ponged, at 16 bytes each: 128 MB apiece. They are sized in nodes rather than threads, so they are subtracted from the memory budget before the per-thread division above.

10 Limits and guards

The program refuses inputs it cannot handle rather than producing a plausible wrong answer. Each guard names the figure that failed.

32-bit bound arithmetic.

The bound machinery is `int32_t` throughout, inherited from the FORTRAN, with 10^9 standing in for infinity. $\sum_{ij} a_{ij} \cdot \max_{kl} b_{kl}$ bounds every objective the instance can produce and so every intermediate the search forms; if that reaches 2^{30} the program stops. This admits the 32×32 `fldata-1024` pair and rejects the 100×100 one.

128-bit frontier ranks.

A rank at level L runs to $n!/(n-L)!$. The expansion stops one level before that would leave 128 bits.

Frontier capacity.

Checked before expanding, as $NF \cdot (n - \text{Depth}) \leq FCap$, so overflow is impossible; the kernel additionally counts past the end without writing so that a violated invariant is reported rather than silently overwriting.

Accumulator width.

$170!$ is the last factorial below 2^{1024} , so $n \leq 170$. Nothing will ever meet this limit.

Expansion depth.

`MAX_FRONTIER_DEPTH` = 32, a backstop on the digit array in `Descend`; the 128-bit rank runs out first at every size where both are reachable.

Small n .

$n \leq 2$ is solved directly on the host: the branch and bound proper needs three free facilities at the root, since its leaves are the nodes with two left.

Note what is *not* on this list. There is no factorial-shaped size cap of the kind `cudaqap` ($n \leq 20$) and `cgbncudaqap` ($n \leq 24$) carry, because the search never forms a rank over the whole permutation space. What remains is a memory limit and a difficulty limit, and the second is a property of the instance rather than of its size: two 25×25 instances in this repository differ by a factor of 4.5 in solution time.

11 Verification

11.1 Oracles

- **QAPLIB.** `chr12a`, `chr20a` and `chr25a` have published optima of 9552, 2192 and 3796. All three are reproduced.
- **qapbb.** The serial branch and bound is the reference implementation of the same bound, on a different tree. It agrees on every instance run to completion, up to and including $N = 30$ (`fldata-900`, 119265331, where `qapbb` takes 2248 s and its own reported assignment re-scores to the same value).
- **qap.** The serial brute force can be run up to $N = 12$. It agrees on cost on all eight small pair-format inputs, and on the assignment as well — which is not guaranteed (Section 7) but is what happened.
- **Self-check.** **Accounted:** must equal $N!$ and **Residual:** must be zero, on every run. The reported assignment is re-scored on the host against the untouched input matrices; the search works in reduced arithmetic, and a disagreement would be reported on standard error. It never has.

11.2 compute-sanitizer

All four tools are clean, and cheap enough that all four can be run on the same instance without planning around them — a consequence of the kernel using no shared memory.

tool	N	GPU time	vs. native	verdict
(none)	12	0.0316 s	1.0×	—
<code>synccheck</code>	12	0.0333 s	1.1×	0 errors
<code>initcheck</code>	12	0.4367 s	14×	0 errors
<code>memcheck</code>	12	0.6704 s	21×	0 errors
(none)	10	0.0145 s	1.0×	—
<code>racecheck</code>	10	0.2322 s	16×	0 hazards, 0 errors, 0 warnings

All four returned the same cost, node count and residual as the uninstrumented run: 953450 in 3567 nodes at $N = 12$, 165908 in 104 nodes at $N = 10$.

11.3 Two bugs the self-check did not catch

Both produced a residual of zero while being wrong, and between them they are the argument for reading the cost, the residual *and* the node count.

The 64-bit frontier rank. The frontier initially held ranks in `uint64_t`. A rank at level L runs to $n!/(n-L)!$, which at $N = 25$ passes 2^{64} at depth 16. A wrapped rank decodes to a different node, so the search covered some subtrees twice and missed others — and reported **Residual:** 0 while doing it, because the accounting faithfully counts what the search disposed of, not what it should have. It returned 3856 for `chr25a` instead of 3796. What caught it was the QAPLIB optimum.

The missing root tables. Both kernels must call `wegspe` once at the root, per thread, to build the level-0 sorted rows that the minimal scalar products are read from. An early version of the rewrite dropped that call. Managed memory happened to be zeroed, so the scalar products came out zero, the bound fell back to its linear part — still a valid lower bound, so still the right answer, and still **Residual:** 0 — and simply stopped pruning: `fldata-400` went from 101 thousand nodes to 88 million. Nothing but the node count showed it.

12 Measurements

Measured on an NVIDIA GeForce RTX 4080 Laptop GPU (`sm_89`, 12282 MiB, driver 575.64.03) against an Intel Core i9-14900HX, with CUDA 12.9 and g++ 14.3.1. `qapbb` times are whole-run; the GPU column is solve time as the program reports it, which excludes parsing and CUDA context setup. Medians of five for the fast instances.

instance	N	minimum cost	GPU nodes	qapbb	cgbncudaqapbb
<code>chr12a</code>	12	9552	1,479	0.0016 s	0.0267 s
<code>fldata-144</code>	12	953450	3,567	0.0016 s	0.0319 s
<code>fldata-225</code>	15	2135449	902	0.0028 s	0.0622 s
<code>fldata-256</code>	16	2942118	17,162	0.0292 s	0.1068 s
<code>chr20a</code>	20	2192	158,236	0.0548 s	0.1677 s
<code>fldata-400</code>	20	9779382	100,978	0.164 s	0.2887 s
<code>chr25a</code>	25	3796	≈7.9 M	11.2 s	2.46 s
<code>fldata-625</code>	25	41048904	8,596,072	71.8 s	2.74 s
<code>fldata-900</code>	30	119265331	33,496,281	2248 s	19.99 s
<code>fldata-1024</code>	32	189919789	1,484,193,064	—	1275.31 s

The crossover is around $N = 20$. Below it the tree is too small to spread over fifteen thousand threads and the GPU is paying setup costs against a search one core finishes in milliseconds; above it the node count grows, and the node count is what parallelises.

The two solvers do not explore the same tree, which the table cannot show. A cleaner parallel-speedup figure comes from a single-threaded host build of `cgbncudaqapbb`'s own algorithm: `fldata-625` 54.0 s against 2.74 s and `fldata-900` 420.7 s against 19.99 s — $20\times$ and $21\times$, on the same tree, the same bound, and very nearly the same node counts (8,596,072 against 8,596,159 at $N = 25$; 33,496,281 against 33,496,339 at $N = 30$).

At $N = 32$ no CPU cross-check exists: `qapbb` was stopped unfinished at two hours and the host build of the n -ary search at one hour thirty-nine, both against the GPU's twenty-one minutes.

12.1 The largest run

```

Minimum cost: 189919789
B&B nodes: 1484193064
Permutations: 263130836933693530167218012160000000
Accounted: 263130836933693530167218012160000000
Residual: 0
Start bound: 189919789
NumThreads: 14848
Frontier: 665914 live node(s) at depth 7 into the depth-first phase
Busiest: 2366935 node(s), 0 idle thread(s)
GPU time: 1275.305643775 second(s).

```

1.48 billion nodes in 21 minutes is 1.16 million bounds a second, each one an LSAP. It is the only instance measured here that reaches the depth-first phase — everything smaller finishes breadth-first — and the busiest of the fifteen thousand threads carried 0.16% of the search. $32!$ is an exact 118-bit answer arrived at by summing 64-bit counts.

13 Algorithm credits

Nothing in this program is original mathematics. Every algorithm it uses is listed below with its origin and the place in the source where it appears. Where a work is cited, the citation is given for attribution; bibliographic details are as commonly cited and were not re-verified against the originals while this was written.

13.1 The problem itself

Quadratic Assignment Problem, Koopmans–Beckmann form.

Tjalling C. Koopmans and Martin Beckmann, 1957. The formulation the entire repository solves, including the linear term C that `RootReduce` generates and the search carries.

Strong NP-hardness and inapproximability.

Sartaj Sahni and Teofilo Gonzalez, 1976. Context rather than code: it is why exact solution is a branch-and-bound problem and not a polynomial-time one.

13.2 Branch and bound

Branch and bound as a method.

Ailsa H. Land and Alison G. Doig, 1960, for the technique; *John D. C. Little, Katta G. Murty, Dura W. Sweeney and Caroline Karel*, 1963, for the name and its popularisation. The whole structure of `ExpandFrontier` and `RunSubtree`.

The Gilmore–Lawler bound.

Paul C. Gilmore, 1962, and *Eugene L. Lawler*, 1963, independently. This is the bound the program computes at every node: reduce, underestimate the quadratic interaction by minimal scalar products, and solve the resulting linear assignment problem. `qapbb::wegspe`, `qapbb::progno_level`, `qapbb::lsap` and `ChildBound` together *are* this bound.

The rearrangement inequality.

G. H. Hardy, J. E. Littlewood and G. Pólya, *Inequalities*, 1934. The theorem underneath the Gilmore–Lawler bound: over all pairings of two finite sequences, the scalar product is minimised by pairing one sorted decreasingly with the other sorted increasingly. It is the entire reason `wegspe` sorts A ’s rows one way and B ’s rows the other, and the reason

`progno_level`'s single pass over the two sorted lists produces a valid lower bound rather than an arbitrary number.

The specific QAP branch-and-bound program.

T. Boenniger, Rainer E. Burkard and Karl-Heinz Stratmann, whose FORTRAN IV implementation (subroutines QAP, ALTKOS, WECSPE, PROGNO, SSORT) is the ancestor of `qapbb.c` and therefore of the bound machinery here. The root reduction, the incremental maintenance of the sorted row tables, the deferred update of the linear term, and the closed-form evaluation of the last two facilities are all theirs, and are used essentially unchanged. ALTKOS and its alternative-cost branching rule are the one part deliberately not carried over (Section 5), and the reason is a property of GPUs, not a criticism of the rule.

13.3 The linear assignment subproblem

The assignment problem and the Hungarian method.

Harold W. Kuhn, 1955, building on work of *Jenő Egerváry* and *Dénes König*; *James Munkres*, 1957, for the polynomial-time formulation. The dual variables `ys/yt` in `qapbb::lsap`, and the row-and-column reduction idea that `RootReduce` also applies to A and B , come from this line.

Shortest augmenting path method for the assignment problem.

Nobuaki Tomizawa, 1971, and *Jack Edmonds and Richard M. Karp*, 1972, for the shortest-augmenting-path formulation with dual updates. This is the algorithm `qapbb::lsap` implements: greedy initial partial assignment, then repeated shortest-path augmentation in the reduced costs.

The particular LSAP implementation.

Ulrich Derigs, whose FORTRAN routine is the one translated in `qapbb.c` and ported here. Its structure — the two-pass greedy start, the L105/L400/L500 labels for shortest path, augment and dual transformation — is preserved exactly.

A primal solution from the relaxation.

A standard branch-and-bound practice with no single originator: the assignment that certifies the bound is itself a feasible solution to the original problem, so scoring it costs almost nothing and often improves the incumbent. `TryCompletion`. In the QAP literature this use of the Gilmore–Lawler assignment is long established.

13.4 Sorting

Shell sort.

Donald L. Shell, 1959. `qapbb::ssort`, ported verbatim including the original increment sequence (start at 1023, halve, ten passes) and the companion-array mechanism. Analysis of the method and its increment sequences is in *Donald E. Knuth, The Art of Computer Programming*, Vol. 3.

13.5 Ranking and unranking permutations

The factorial number system.

Charles-Ange Laisant, 1888, for the mixed-radix numeration applied to permutations. The frontier rank $\text{rank}_L = \text{rank}_{L-1}(n - k) + m$ is exactly a factorial-base number truncated to L digits, and `Descend` decodes it by the standard repeated division.

The Lehmer code.

Derrick H. Lehmer, 1960, for the code itself and for its use in machine enumeration of permutations. Treatment in *Knuth*, TAOCP Vol. 4A. The repository’s brute-force solvers use full-permutation ranking from the same idea; this program uses the truncated form for partial permutations.

13.6 Heuristics

2-opt / pairwise exchange local search.

Georges A. Croes, 1958, for the travelling salesman problem; *Shen Lin*, 1965, for the systematic k -opt treatment. Applied to the QAP the move is a transposition of two facilities’ locations, which is what `StartBound` does, from the identity permutation and its cyclic shifts, by first improvement.

Ordering the branching variables by connectivity.

A folk heuristic of the constraint and integer programming literature — branch on the most constrained variable first — specialised here to “place the highest-flow facility first”. `ReorderFacilities`.

13.7 Parallel search

Parallel branch and bound and its anomalies.

Ten-Hwang Lai and Sartaj Sahni, 1984, for the analysis of why parallel branch and bound does not simply scale — speculative work, detrimental anomalies, and the dependence of the node count on timing. The non-reproducible node count described in Section 7 is precisely this phenomenon, and the design of Section 6 is a response to it.

Persistent threads with a global work queue.

Timo Aila and Samuli Laine, 2009, who introduced the pattern for ray traversal; studied systematically by *Kshitij Gupta, Jeff A. Stuart and John D. Owens*, 2012. `BranchAndBound`’s `atomicAdd` on a single device-wide cursor is that pattern.

Breadth-first frontier expansion before depth-first search on a GPU.

The standard shape of GPU branch and bound in the literature — expand a level at a time until there is enough independent work, then switch to depth-first per subtree — as developed for permutation problems by *Imen Chakroun, Nouredine Melab, Mohand Mezmaiz and Daniel Tuytens*, 2013, and *Jan Gmys, Mohand Mezmaiz, Nouredine Melab and Daniel Tuytens*, 2016, among others. `ExpandFrontier` plus `BranchAndBound` is an instance of it, with a rank-based node representation in place of their integer-vector-matrix structure.

13.8 Arbitrary-precision arithmetic

CGBN — Cooperative Groups Big Numbers.

NVIDIA Labs; a fixed-width multiple-precision library for CUDA in which one value’s limbs are distributed across TPI cooperating lanes of a warp. `AccumulateIterations`, `CombineIterations` and `CombinePermutations` are written against it. The `cgbn_signed.*` extension used for the residual comes from the fork installed on this machine.

GNU Multiple Precision Arithmetic Library (GMP).

Torbjörn Granlund and the GMP development team. Used on the host to build the table of factorials in CGBN’s limb layout (`SetupFactorials`, via `mpz_export`) and to convert accumulators to decimal (`BigToString`, via `mpz_import` and `mpz_get_str`). GMP is not optional: `cgbn.h` is compiled on the host pass as well as the device pass and selects its host backend from whether `<gmp.h>` has been seen.

13.9 Data and tools

QAPLIB.

Rainer E. Burkard, Stefan E. Karisch and Franz Rendl, 1997. The source of the chr instances and of the published optima used as the correctness oracle.

The chr instance family.

Nicos Christofides and Enrique Benavent, 1989, who constructed these tree-structured instances.

CUDA, nvcc, and the occupancy API.

NVIDIA. `cudaOccupancyMaxActiveBlocksPerMultiprocessor` chooses the block width; `compute-sanitizer` provided the memory, race, synchronisation and initialisation checks reported in Section 11.

13.10 From elsewhere in this repository

The packed argmin key.

From `cudaqap.cu`: `pack (cost << 32) | threadIdx` so that one 64-bit `atomicMin` both selects the minimum and breaks ties deterministically. Here it does double duty as the shared pruning threshold.

The launch-geometry search and its two traps.

From `cudaqap.cu`: never call `cudaGetDeviceProperties` on the setup path, and never select a candidate width reporting zero resident blocks.

The CGBN accumulator pattern and 128-bit ranks.

From `cgbncudaqap.cu`.

The option handling, readers and printers.

From `qap.cpp` through `cudaqap.cu`.

14 References

The list below is for attribution. Details are as commonly cited; they were not re-verified against the original publications while this document was written, and a reader relying on a specific page or volume number should check it.

1. T. Aila and S. Laine. Understanding the efficiency of ray traversal on GPUs. *Proc. High Performance Graphics*, 2009.
2. R. E. Burkard, S. E. Karisch and F. Rendl. QAPLIB — a quadratic assignment problem library. *Journal of Global Optimization*, 10(4):391–403, 1997.
3. R. E. Burkard and K.-H. Stratmann. Numerical investigations on quadratic assignment problems. *Naval Research Logistics Quarterly*, 25(1):129–148, 1978.
4. I. Chakroun, N. Melab, M. Mezmaz and D. Tuytens. Combining multi-core and GPU computing for solving combinatorial optimization problems. *Journal of Parallel and Distributed Computing*, 73(12):1563–1577, 2013.
5. N. Christofides and E. Benavent. An exact algorithm for the quadratic assignment problem on a tree. *Operations Research*, 37(5):760–768, 1989.
6. G. A. Croes. A method for solving traveling-salesman problems. *Operations Research*, 6(6):791–812, 1958.

7. U. Derigs. A shortest augmenting path method for solving minimal perfect matching problems. *Networks*, 11(4):379–390, 1981.
8. J. Edmonds and R. M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972.
9. P. C. Gilmore. Optimal and suboptimal algorithms for the quadratic assignment problem. *Journal of the Society for Industrial and Applied Mathematics*, 10(2):305–313, 1962.
10. J. Gmys, M. Mezma, N. Melab and D. Tuytens. A GPU-based branch-and-bound algorithm using integer-vector-matrix data structure. *Parallel Computing*, 59:119–139, 2016.
11. T. Granlund and the GMP development team. *GNU MP: The GNU Multiple Precision Arithmetic Library*. <https://gmplib.org/>
12. K. Gupta, J. A. Stuart and J. D. Owens. A study of persistent threads style GPU programming for GPGPU workloads. *Innovative Parallel Computing (InPar)*, 2012.
13. G. H. Hardy, J. E. Littlewood and G. Pólya. *Inequalities*. Cambridge University Press, 1934.
14. D. E. Knuth. *The Art of Computer Programming*, Vol. 3: *Sorting and Searching*, and Vol. 4A: *Combinatorial Algorithms, Part 1*. Addison-Wesley.
15. T. C. Koopmans and M. Beckmann. Assignment problems and the location of economic activities. *Econometrica*, 25(1):53–76, 1957.
16. H. W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
17. T.-H. Lai and S. Sahni. Anomalies in parallel branch-and-bound algorithms. *Communications of the ACM*, 27(6):594–602, 1984.
18. C.-A. Laisant. Sur la numération factorielle, application aux permutations. *Bulletin de la Société Mathématique de France*, 16:176–183, 1888.
19. A. H. Land and A. G. Doig. An automatic method of solving discrete programming problems. *Econometrica*, 28(3):497–520, 1960.
20. E. L. Lawler. The quadratic assignment problem. *Management Science*, 9(4):586–599, 1963.
21. D. H. Lehmer. Teaching combinatorial tricks to a computer. *Proceedings of Symposia in Applied Mathematics*, Vol. 10, American Mathematical Society, 179–193, 1960.
22. S. Lin. Computer solutions of the traveling salesman problem. *Bell System Technical Journal*, 44(10):2245–2269, 1965.
23. J. D. C. Little, K. G. Murty, D. W. Sweeney and C. Karel. An algorithm for the traveling salesman problem. *Operations Research*, 11(6):972–989, 1963.
24. J. Munkres. Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1):32–38, 1957.
25. NVIDIA Labs. *CGBN: CUDA Accelerated Multiple Precision Arithmetic (Big Num) using Cooperative Groups*. <https://github.com/NVlabs/CGBN>

26. S. Sahni and T. Gonzalez. P-complete approximation problems. *Journal of the ACM*, 23(3):555–565, 1976.
27. D. L. Shell. A high-speed sorting procedure. *Communications of the ACM*, 2(7):30–32, 1959.
28. N. Tomizawa. On some techniques useful for solution of transportation network problems. *Networks*, 1(2):173–194, 1971.

A Source map

function	where	role and origin
<code>qapbb::ssort</code>	device	Shell sort with companion array. Shell 1959, via <code>qapbb.c</code> .
<code>qapbb::lsap</code>	device	LSAP by shortest augmenting path. Tomizawa / Edmonds–Karp; Derigs’ implementation, via <code>qapbb.c</code> .
<code>qapbb::wegspe</code>	device	Sorted row tables per level. Boenniger–Burkard–Stratmann; rearrangement inequality.
<code>qapbb::progno_level</code>	device	Minimal scalar products. Same, with the CSPEI cache removed.
<code>RootReduce</code>	host	Reduction of A, B into C . Same.
<code>ReorderFacilities</code>	host	Relabel by decreasing total flow. Branching-order heuristic.
<code>StartBound</code>	host	2-opt from identity and cyclic shifts. Croes 1958.
<code>HostObjective,</code> <code>WorkObjective</code>	host	Objective evaluation; the re-score check.
<code>SetupFactorials</code>	host	$0! \dots N!$ into CGBN limb layout. GMP.
<code>BigToString</code>	host	CGBN to decimal. GMP.
<code>LayoutWork</code>	both	Per-thread workspace layout; one function for host sizing and device carving.
<code>Descend</code>	device	Rebuild a node’s state from its mixed-radix rank. Laisant / Lehmer.
<code>ChildBound</code>	device	One node’s Gilmore–Lawler bound. Gilmore 1962, Lawler 1963.
<code>TryCompletion</code>	device	Score the LSAP’s own assignment as a feasible solution.
<code>LeafPair</code>	device	Closed form for the last two facilities. <code>qapbb.c</code> label 5330.
<code>ShiftC</code>	device	Fold a placed pair into, or out of, the linear term. <code>qapbb.c</code> label 5220.
<code>ExpandFrontier</code>	kernel	One level of breadth-first expansion.
<code>RunSubtree</code>	device	Depth-first search of one frontier node’s subtree.
<code>BranchAndBound</code>	kernel	Persistent threads over the frontier work queue. Aila–Laine; Gupta et al.
<code>AccumulateIterations,</code> <code>CombineIterations</code>	kernel	Two-stage CGBN sum of the node counts.
<code>SumCounters</code>	kernel	Per-level disposal counts, 64-bit atomics.
<code>CombinePermutations</code>	kernel	$\sum_L \text{count}_L (N - L)!$ and the signed residual. CGBN.

B Output fields

field	meaning
<code>Minimum cost:</code>	The optimum, computed by <code>HostObjective</code> from the untouched input matrices for the reported assignment.
<code>B&B nodes:</code>	Bounds computed, summed in 1024-bit CGBN. Not reproducible run to run (Section 7).
<code>Permutations:</code>	$N!$, computed on the host by GMP.
<code>Accounted:</code>	$\sum_L \text{count}_L(N - L)!$ over disposed nodes. Must equal <code>Permutations:</code> .
<code>Residual:</code>	<code>Permutations:</code> - <code>Accounted:</code> , signed. Must be 0.
<code>Start bound:</code>	The host 2-opt bound. Equal to the reported cost means the heuristic was already optimal and the GPU work was the proof.
<code>NumThreads:</code>	Threads actually launched.
<code>Frontier:</code>	Live nodes handed to the depth-first phase, and the depth they were cut at. - <code>solved breadth-first</code> means the expansion finished the search on its own.
<code>Busiest:</code>	Nodes in the thread that did the most, and how many threads did nothing. A <code>Busiest</code> approaching <code>B&B nodes</code> means the frontier was cut too shallow.
<code>GPU time:</code>	Frontier expansion plus depth-first phase. Excludes parsing, the host heuristic, and CUDA context setup.
<code>CGBN reduction:</code>	The four accounting kernels, timed apart from the search.